

through the configuration of the APIs being invoked rather than writing the code from the ground up, thus saving time.

While the majority of AI-specific solutions are written in Python and R, a lot of SDKs allow developers to create and/or consume ML models in .NET Core (Microsoft's open source version of .NET), Java, JavaScript and other languages. This presents a significant opportunity for application developers to leverage these platforms to learn about ML and AI through their favourite languages.

Dotnet Core and machine learning

Dotnet Core (aka .NET Core) is a free, cross-platform, open source developer platform for building many different types of applications. It offers support for writing apps in Windows, Linux and Mac OS on mobile devices, the Web, for gaming, IoT and ML in a language of your choice: C#, F# or Visual Basic. These languages have been around for a while and continue to be popular among developer audiences as application development platforms. While the use of .NET has been predominantly on the frontend/backend or middleware layers of a solution, the introduction of .NET for ML creates a new dimension on how application development platforms are evolving.

If you are skilled in open source platforms or have always been a .NET developer on Windows, this is the right time for you to start exploring this cross-platform, open source platform through proven programming languages in the stack.

While AI solutions have mostly been developed in Python and R, .NET Core introduces a new paradigm where the development and use of models is powered by a framework called ML.NET. The framework also has built-in implementations of popular algorithms like logistics, linear regressions, SVM and many others that are commonly used in the field of ML for model development.

ML.NET enables you to develop and integrate custom ML models into your applications even while you navigate through the basics of ML. Being an open source cross-platform framework for .NET developers, ML.NET is an extensible platform that powers Microsoft features like Windows Hello, Bing Ads, PowerPoint Design Ideas and more. This article focuses on ML.NET version 0.6.

ML.NET capabilities and features

There are primarily three tasks in any ML experiment (or application).

1. Data processing
2. Creating a training workflow
3. Creating a scoring workflow

ML.NET provides namespaces and classes to achieve the above tasks through built-in classes and functions. For example, Microsoft.ML.Runtime.Data and Microsoft.ML.Core.Data have classes that help you to extract, load and transform data into the memory of the program for further processing by the ML engine. Similarly, there are classes in the Microsoft.ML.StaticPipe namespace which help you

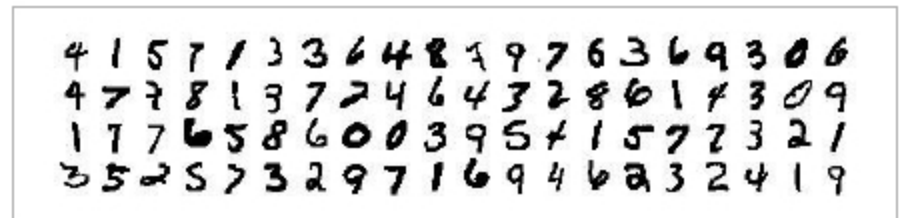


Figure 1: Images in the MNIST data set

to create a pipeline for activities and configurations related to the ML aspects of AI. Finally, there are classes that help you to score or predict the basis of the developed model and get an output. There are also classes within the Microsoft.ML.Runtime.Data namespace that help you to output the performance metrics of the trained model. These metrics help you to understand how the model has been able to perform against training and test data. For example, to measure the performance of a binary logistic regression classifier (a model which outputs only two classes), we study the attribute values of 'F1 Score', 'Accuracy', 'AUC', etc.

Given that ML.NET is built on the strong fundamentals and the same platform as other .NET Core technologies, you can use this inside any of your .NET Core apps like the console, Web (ASP.NET Core) and class libraries or apps running inside Docker containers on Linux.

Time to get your hands dirty!

The ML.NET documentation on Microsoft Docs has a few walk-throughs to get started with some of the scenarios. We will look at using the Logistic Regression classifier on the MNIST data set.

The MNIST data set is like the 'Hello World!' of your machine learning journey. In almost any ML technology that you pick, you will come across an exercise that helps you understand the process using this data set.

The MNIST data set contains handwritten images of digits, ranging from 0 to 9. An example of what the source images look like is shown in Figure 1.

Identifying the digit in an image is a classification task and there are algorithms such as the Random Forest classifier or the Naïve Bayes classifier which can easily deal with multi-class classification. On the other hand, Logistic Regression is a binary classifier at the core, and it gives a binary outcome between 0 and 1 by measuring the relationship between the label and one or more features in the data set. Being a simple and easy-to-implement algorithm, you can use it to benchmark your ML tasks.

Prima facie, being a binary classifier, it doesn't seem suitable for MNIST data set classification, but when it is trained using the One-vs-All (OvA) method, you get a distinct classifier that can identify a specific class from the data set. When you perform the prediction, you just need to find out which classifier returns the best score to identify the digit on the image.

Preparing yourself

If you have not yet installed the .NET Core framework, do